

Mahavir Polytechnic, Nashik
Department of Computer Engineering

Year: TY

Subject: STE (315323)

UNIT 4: Software Project Cost Estimation**Marks: 12****COURSE LEVEL LEARNING OUTCOME:**

CO4	Apply different planning and cost estimation techniques for a software product.
------------	--

THEORY LEARNING OUTCOMES (TLO'S) ALIGNED TO COS:

TLO 4.1	Explain the management spectrum for software project.
TLO 4.2	Estimate size of software product.
TLO 4.3	Estimate cost of software product using the empirical method.
TLO 4.4	Compute size of the given software using COCOMO model.
TLO 4.4	Apply RMMM strategy in Identified risks for any software development problem.

LEARNING CONTENT MAPPED WITH THEORY LEARNING OUTCOMES (TLO'S) AND COS:

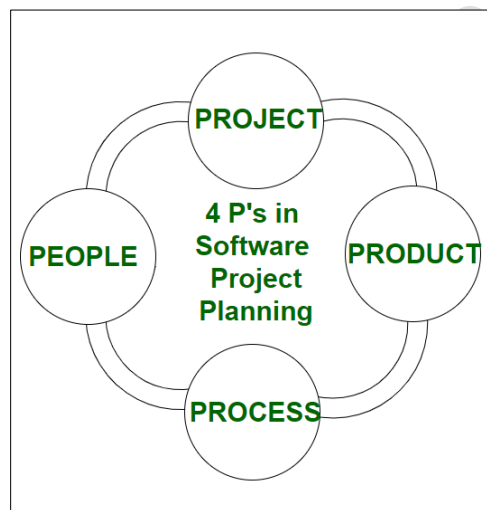
TLO 4.1	The Management Spectrum – 4P's.
TLO 4.2	Metrics for Size Estimation: Line of Code (LoC), Function Points (FP).
TLO 4.3	Project Cost Estimation Approaches: Overview of Heuristic, Analytical, and Empirical Estimation.
TLO 4.4	COCOMO (Constructive Cost Model), COCOMO II.

TLO 4.4	Risk Analysis and Management: Risk identification, Risk assessment, Risk management and monitoring, Risk Refinement and Mitigation, RMMM Plan.
----------------	---

➤ **TLO 4.1:**

➤ **Explain the management spectrum for software project.**

- These components play a very important role in your project that can help your team meet its goals and objective.
- The Management Spectrum for software projects centers on four essential components—People, Product, Process, and Project.



1. People:

The most important component of a product and its successful implementation is human resources. In building a proper product, a well-managed team with clear-cut roles defined for each person/team will lead to the success of the product. We need to have a good team in order to save our time, cost, and effort. Some assigned roles in software project planning are project manager, team leaders, stakeholders, analysts, and other IT professionals. Managing people successfully is a tricky process which a good project manager can do.

2. Product

As the name inferred, this is the deliverable or the result of the project. The project manager should clearly define the product scope to ensure a successful result, control the team members, as well technical hurdles that he or she may encounter during the building of a product. The product can consist of both tangible or intangible such as shifting the company to a new place or getting a new software in a company.

3. Process

In every planning, a clearly defined process is the key to the success of any product. It regulates how the team will go about its development in the respective time period. The Process

has several steps involved like, documentation phase, implementation phase, deployment phase, and interaction phase.

4. Project

The last and final P in software project planning is Project. It can also be considered as a blueprint of process. In this phase, the project manager plays a critical role. They are responsible to guide the team members to achieve the project's target and objectives, helping & assisting them with issues, checking on cost and budget, and making sure that the project stays on track with the given deadlines.

➤ **TLO 4.2:**

➤ **Estimate size of software product.**

- Estimation of the size of the software is an essential part of Software Project Management. It helps the project manager to further predict the effort and time that will be needed to build the project.
- Here are some of the measures that are used in project size estimation:
- Software product size estimation determines the scale and scope of a project by quantifying its functionality, such as through Lines of Code (LOC) or Function Points (FP).
- This initial size measurement is crucial for planning, as it directly influences subsequent estimates for project effort, resources, schedule, and cost.
- Techniques vary, from breaking down the project into smaller modules to using expert judgment and comparing with past projects.

➤ **Why Size Estimation is Important**

- **Basis for other estimates:**

A clear understanding of the software's size helps in more accurately predicting the effort, time, and cost required for its development.

- **Project planning:**

Size estimation provides a foundational metric for project managers to create a realistic project plan and manage expectations.

- **Tracking and control:**

It allows for progress tracking and control throughout the development lifecycle.

- **Resource allocation:**

Knowing the size helps in determining the necessary human resources and tools for the project.

➤ **Key Metrics for Size Estimation**

- **Lines of Code (LOC):**

Measures the total number of lines in the source code, which can be quantified as KLOC (thousands of lines of code).

- **Function Points (FP):**

Quantifies the complexity of the software based on the number of functions, inputs, outputs, files, and interfaces it provides to the user. This metric is language-independent and focuses on the functionality offered to the user.

➤ **Common Estimation Techniques**

- **Analogous Estimation:**

Compares the current project with similar projects that have been developed in the past to estimate its size.

- **Expert Judgment:**

Leverages the experience of experts to provide estimates based on their knowledge of similar systems and functionalities.

- **Work Breakdown Structure (WBS):**

Divides the software project into smaller, manageable sub-systems and modules, allowing for size estimation at a more detailed level.

- **Parametric Models:**

Uses mathematical models based on historical data and project parameters to predict size.

Factors Influencing Size Estimation

- **Project Complexity:** More complex projects require more effort and are thus larger in scope.
- **Requirements and Functionality:** The number and complexity of features defined in the Software Requirements Specification (SRS) document significantly impact the project size.
- **Team Experience and Technology:** The skills of the development team and the chosen development technologies can affect the final size and the efficiency of the estimation process.

4.2.1 LOC (Line of Code):

Lines of Code (LOC) is a software metric used to measure the size of a computer program by counting the number of lines in the text of the program's source code. It is one of the oldest and most basic metrics in software engineering.

➤ **Methods of Measuring LOC:**

- **Physical Lines of Code (PLOC):**

This method counts all lines in the source code, including comments, blank lines, and declaration statements.

- **Logical Lines of Code (LLOC):**

This method focuses on executable statements, excluding comments, blank lines, and often declarations, aiming to represent the functional logic.

➤ **Uses of LOC:**

- **Size Estimation:**

LOC can be used to estimate the size of a software project, which in turn can help in estimating development effort, cost, and schedule.

- **Comparison:**

It can be used to compare the size of different software projects or different versions of the same project, particularly when using the same programming language and coding standards.

- **Productivity Measurement (with caveats):**

Historically, LOC has been used as a measure of developer productivity, though this use is widely criticized due to its limitations.

➤ **Limitations of LOC:**

- **Language Dependence:**

Different programming languages can achieve the same functionality with varying numbers of lines of code.

- **Coding Style and Formatting:**

Variations in coding style, such as placing multiple statements on one line or using extensive whitespace, can significantly affect LOC counts without changing functionality.

- **Quality and Complexity:**

LOC does not inherently reflect the quality, complexity, maintainability, or efficiency of the code. A large number of lines does not necessarily mean a better or more functional program.

- **Discourages Refactoring:**

Over-reliance on LOC as a metric can discourage developers from refactoring and writing concise, efficient code, as it might appear to reduce their "productivity."

While LOC can provide a basic indication of code volume, it is generally recommended to use it in conjunction with other software metrics, such as cyclomatic complexity, function points, or more modern DORA metrics, for a more comprehensive understanding of software characteristics and project performance.

4.2.2 Function-Points (FP)

- In software engineering, a function point (FP) is a unit of measurement that quantifies the business functionality delivered by a software application, used to estimate project size, effort, cost, and schedule.
- FPA involves counting five types of functions—external inputs, external outputs, external inquiries, internal logical files, and external interface files—assigning them complexity weights (simple, average, complex), and summing these to get an unadjusted function point total.
- This is then adjusted by 14 general system characteristics to get the final FP count, providing a standardized and user-focused metric for project planning and analysis.

➤ How Function Points Work

1. Identify Functional Components:

The process begins by identifying and categorizing the software's functions, which fall into five main types:

- **External Inputs (EI):** Data or transactions from users or external systems.
- **External Outputs (EO):** Derived data that provides information to users, such as reports or screens.
- **External Inquiries (EQ):** Inputs that result in a software response or output.
- **Internal Logical Files (ILF):** Logical groupings of data maintained within the application.
- **External Interface Files (EIF):** Logical groupings of data external to the application but used by it.

2. Assign Complexity:

Each identified component is assigned a complexity rating (e.g., simple, average, or complex) based on predefined criteria.

3. Calculate Unadjusted Function Points (UFP):

The number of each type of function is multiplied by its assigned complexity weight to get the component's value, and these are summed to produce the UFP.

4. Apply a Value Adjustment Factor (VAF):

The UFP is then adjusted by a VAF, which accounts for 14 general system characteristics (GSCs) that affect the system's overall complexity. Each GSC is rated for its importance, and the total score is used to calculate the VAF.

5. Determine Final Function Points:

The UFP is multiplied by the VAF to yield the final function point count.

Why Function Points Are Used

- **User-Centric:**

FPA measures functionality from the end-user's perspective, making it easier to understand and discuss with stakeholders.

- **Standardized & Objective:**

It provides a uniform, objective measure of software size independent of the implementation language or technology used.

- **Early Estimation:**

FPA can be performed in the early stages of a project, even from requirements, making it useful for early-stage cost and schedule estimates.

- **Project Management:**

The resulting FP count is used to forecast costs, effort, and timelines, enabling better resource planning.

- **Benchmarking:**

FPA allows for comparison and analysis of development performance over time and across different projects.

4.3 Estimate cost of software product using the empirical method:

- To estimate the cost of a software product using the empirical method, leverage data from past projects, consider factors like project size and complexity, and use established techniques such as Expert Judgment and the Delphi Technique or algorithmic models like COCOMO.
- The empirical method relies on educated guesses and historical data to predict effort and development time, a critical step in software project planning.

➤ **Key Steps for Empirical Cost Estimation**

- 1. Gather Historical Data:**

Collect data from previous software projects to build a knowledge base about development effort, time, cost, and project characteristics.

- 2. Analyze Project Requirements:**

Thoroughly analyze the new software product's specifications and scope to understand its complexity and requirements.

- 3. Break Down the Project:**

Divide the overall project into smaller, more manageable components or tasks for independent estimation.

4. **Select an Empirical Technique:**

Choose an appropriate technique based on the project's characteristics and the available data.

Popular Empirical Techniques

➤ **Expert Judgment:**

An experienced expert or a group of experts provides an educated guess of the project's cost and effort after a thorough analysis of the problem and its components.

➤ **Delphi Technique:**

A systematic process involving a panel of experts who provide their cost estimates anonymously. A coordinator collects these estimates and presents the summarized results back to the experts, who then revise their estimates until a consensus is reached.

➤ **Algorithmic Models (e.g., COCOMO):**

These models use empirically derived formulas and statistical techniques to predict software cost, effort, and time based on project size and other factors.

Factors Influencing Empirical Estimation

- **Project Size:** The estimated number of lines of code (SLOC) or other size metrics.
 - **Complexity:** The difficulty of the software product and its features.
 - **Development Environment:** The experience level of the development team and the rigidity of the development environment.
 - **Project Scope:** The defined boundaries and features of the software product.
-

➤ **TLO 4.4**

➤ **Compute size of the given software using COCOMO model:**

- The Constructive Cost Model (COCOMO) It was proposed by Barry Boehm in 1981 and is based on the study of 63 projects, which makes it one of the best-documented models.
- It is a Software Cost Estimation Model that helps predict the effort, cost, and schedule required for a software development project.
- To calculate a software's size with the COCOMO model, you first estimate the size in Kilo Lines of Code (KLOC) or Kilo Delivered Source Instructions (KDSI), then use this to calculate the development Effort (in person-months) and Development Time (in months) with formulas like $E = a * (KLOC)^b$ and $T_{dev} = c * (E)^d$.

- The specific values for the constants (a, b, c, d) depend on the project's development mode (organic, semi-detached, or embedded) and must be selected from a given table.

➤ **Steps to Calculate Software Size using COCOMO:**

1. Determine the Project Type and Select Constants:

- Based on the estimated size and project characteristics, classify the project into one of the COCOMO modes:

➤ **Organic:**

- Small, well-understood projects with experienced developers in a familiar environment.
- A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

➤ **Semi-detached:**

- Medium-sized projects with a mix of experienced and new developers or a mix of familiar and innovative modules.
 - A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environments lie in between organic and embedded.
 - The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience better guidance and creativity. Eg: Compilers or different Embedded Systems can be considered Semi-Detached types.

➤ **Embedded:**

- Large, complex projects requiring significant innovation, tight deadlines, and dealing with intricate interfaces and real-time systems.
- Find a COCOMO table (like the example in the search results) that lists the coefficients (a, b, c, d) corresponding to the determined project mode.

2. Estimate Software Size (KLOC/KDSI):

- Get an early estimate of the project's size in Kilo Lines of Code (KLOC) or Kilo Delivered Source Instructions (KDSI). This is the starting point for the COCOMO calculation.

3. Calculate Effort (E):

- Use the Basic COCOMO formula: $E = a * (KLOC)^b$.
- Where:
- E is the total effort in person-months (PM).

- a and b are the constants for your chosen project mode.
- KLOC is the estimated software size.

4. Calculate Development Time (Tdev):

- Use the formula: $Tdev = c * (E)^d$.
- Where:
- Tdev is the development time in months.
- c and d are the constants for your chosen project mode.
- E is the calculated Effort.

5. Calculate Number of Personnel (Optional):

- The number of people required can be calculated by dividing the total effort by the development time:
 $Personnel = E / Tdev$.

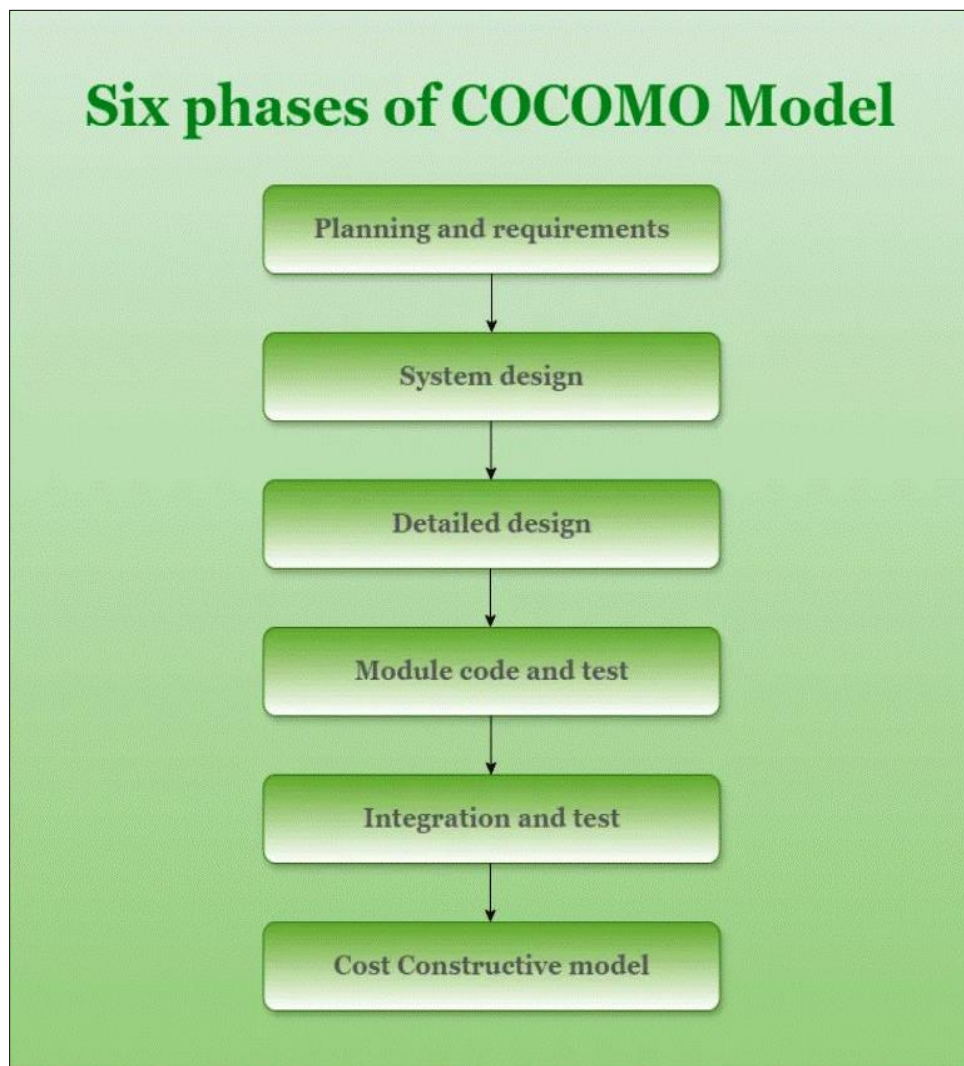
➤ **Comparison of Types of Projects in COCOMO Model**

Here is the Comparison in detail where the project types of COCOMO Model:

Aspects	Organic	Semidetached	Embedded
Project Size	2 to 50 KLOC	50-300 KLOC	300 and above KLOC
Complexity	Low	Medium	High
Team Experience	Highly experienced	Some experienced as well as inexperienced staff	Mixed experience, includes experts
Environment	Flexible, fewer constraints	Somewhat flexible, moderate constraints	Highly rigorous, strict requirements
Effort Equation	$E = 2.4(400)^{1.05}$	$E = 3.0(400)^{1.12}$	$E = 3.6(400)^{1.20}$
Example	Simple payroll system	New system interfacing with existing systems	Flight control software

4.4.1 Structure of COCOMO Model

- Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the Software Engineering Process.
- In detailed COCOMO, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort.
- The Six phases of detailed COCOMO are:



1. **Planning and requirements:** This initial phase involves defining the scope, objectives, and constraints of the project. It includes developing a project plan that outlines the schedule, resources, and milestones
2. **System design:** : In this phase, the high-level architecture of the software system is created. This includes defining the system's overall structure, including major components, their interactions, and the data flow between them.

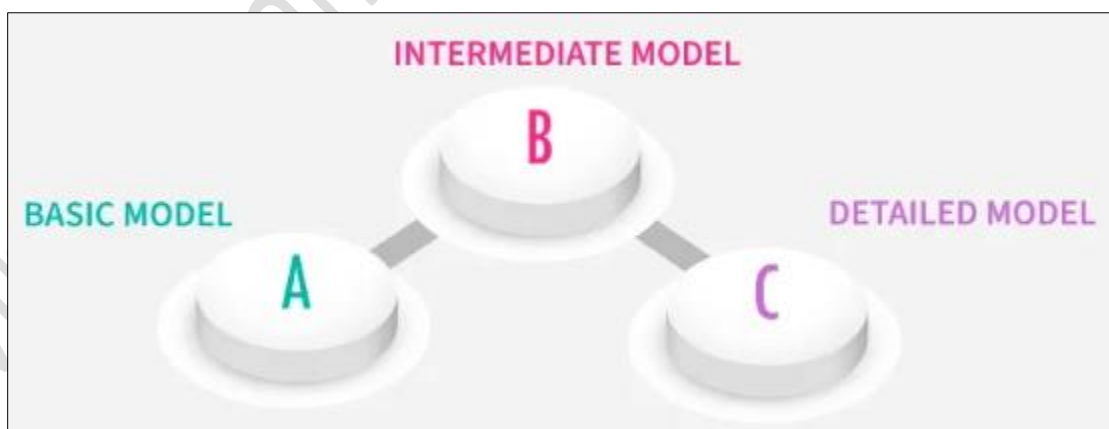
3. **Detailed design:** This phase involves creating detailed specifications for each component of the system. It breaks down the system design into detailed descriptions of each module, including data structures, algorithms, and interfaces.
4. **Module code and test:** This involves writing the actual source code for each module or component as defined in the detailed design. It includes coding the functionalities, implementing algorithms, and developing interfaces.
5. **Integration and test:** This phase involves combining individual modules into a complete system and ensuring that they work together as intended.
6. **Cost Constructive model:** The Constructive Cost Model (COCOMO) is a widely used method for estimating the cost and effort required for software development projects.

➤ **Importance of the COCOMO Model**

1. **Cost Estimation:** To help with resource planning and project budgeting, COCOMO offers a methodical approach to software development cost estimation.
2. **Resource Management:** By taking team experience, project size, and complexity into account, the model helps with efficient resource allocation.
3. **Project Planning:** COCOMO assists in developing practical project plans that include attainable objectives, due dates, and benchmarks.
4. **Risk management:** Early in the development process, COCOMO assists in identifying and mitigating potential hazards by including risk elements.
5. **Support for Decisions:** During project planning, the model provides a quantitative foundation for choices about scope, priorities, and resource allocation.
6. **Benchmarking:** To compare and assess various software development projects to industry standards, COCOMO offers a benchmark.
7. **Resource Optimization:** The model helps to maximize the use of resources, which raises productivity and lowers costs.

➤ **Types of COCOMO Model**

There are three types of COCOMO Model:



1. Basic COCOMO Model

- The Basic COCOMO model is a straightforward way to estimate the effort needed for a software development project.

- It uses a simple mathematical formula to predict how many person-months of work are required based on the size of the project, measured in thousands of lines of code (KLOC).
- It estimates effort and time required for development using the following expression:
 - $E = a \cdot (KLOC)^b$ PM
 - $T_{dev} = c \cdot (E)^d$
 - Person required = Effort/ Time
 - Where,
 - E is effort applied in Person-Months
 - KLOC is the estimated size of the software product indicate in Kilo Lines of Code
 - T_{dev} is the development time in months
 - a, b, c are constants determined by the category of software project given in below table.
- The above formula is used for the cost estimation of the basic COCOMO model and also is used in the subsequent models.
- The constant values a, b, c , and d for the Basic Model for the different categories of the software projects are:

Software Projects	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

- The effort is measured in Person-Months and as evident from the formula is dependent on Kilo-Lines of code. The development time is measured in months.
- These formulas are used as such in the Basic Model calculations, as not much consideration of different factors such as reliability, and expertise is taken into account, henceforth the estimate is rough.

Example of Basic COCOMO Model:

Suppose that a Basic project was estimated to be 400 KLOC (kilo lines of code). Calculate effort and time for each of the three modes of development. All the constants value provided in the following table:

Solution: From the above table we take the value of constant a, b, c and d .

- For organic mode,
 - effort = $2.4 \times (400)^{1.05} \approx 1295$ person-month.
 - dev. time = $2.5 \times (1295)^{0.38} \approx 38$ months.
- For semi-detach mode,
 - effort = $3 \times (400)^{1.12} \approx 2462$ person-month.

- dev. time = $2.5 \times (2462)^{0.35} \approx 38$ months.
3. For Embedded mode,
- effort = $3.6 \times (400)^{1.20} \approx 4772$ person-month.
 - dev. time = $2.5 \times (4772)^{0.32} \approx 38$ months.

➤ **Below are the programs for Basic COCOMO Model:**

```
// C++ program to implement basic COCOMO
#include <bits/stdc++.h>
using namespace std;
// Function For rounding off float to int
int fround(float x)
{
    int a;
    x = x + 0.5;
    a = x;
    return (a);
}
// Function to calculate parameters
// of Basic COCOMO
void calculate(float table[][4], int n, char mode[][15], int size)
{
    float effort, time, staff;
    int model;
    // Check the mode according to size
    // organic
    if (size >= 2 && size <= 50)
        model = 0;
    // semi-detached
    else if (size > 50 && size <= 300)
        model = 1;
    // embedded
    else if (size > 300)
        model = 2;
    cout << "The mode is " << mode[model];
    // Calculate Effort
    effort = table[model][0] * pow(size, table[model][1]);

    // Calculate Time
    time = table[model][2] * pow(effort, table[model][3]);

    // Calculate Persons Required
    staff = effort / time;

    // Output the values calculated
    cout << "\nEffort = " << effort << " Person-Month";

    cout << "\nDevelopment Time = " << time << " Months";
```

```
    cout << "\nAverage Staff Required = " << fround(staff) << " Persons";
}

// Driver code
int main()
{
    float table[3][4] = {2.4, 1.05, 2.5, 0.38, 3.0, 1.12, 2.5, 0.35, 3.6, 1.20, 2.5, 0.32};
    char mode[][15] = {"Organic", "Semi-Detached", "Embedded"};
    int size = 4;
    calculate(table, 3, mode, size);
    return 0;
}
```

OUTPUT

```
The mode is Organic
Effort = 10.289 Person-Month
Development Time = 6.06237 Months
Average Staff Required = 2 Persons
```

2. Intermediate COCOMO Model

The basic COCOMO model assumes that the effort is only a function of the number of lines of code and some constants evaluated according to the different software systems. However, in reality, no system's effort and schedule can be solely calculated based on Lines of Code. For that, various other factors such as reliability, experience, and Capability. These factors are known as **Cost Drivers (multipliers)** and the Intermediate Model utilizes 15 such drivers for cost estimation.

Classification of Cost Drivers and their Attributes:

The cost drivers are divided into four categories

Product attributes:

- Required Software Reliability extent
- Size of the application database
- The complexity of the product

Hardware attributes:

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

Personal attributes:

- Analyst capability
- Software engineering capability
- Application experience
- Virtual machine experience
- Programming language experience

Project attributes:

- Use of Software Tools.
- Application of Software Engineering Methods.
- Required development schedule.

Each of the 15 such attributes can be rated on a six-point scale ranging from "very low" to "extra high" in their relative order of importance. Each attribute has an effort multiplier fixed as per the rating.

Table give below represents Cost Drivers and their respective rating:

*The **Effort Adjustment Factor (EAF)** is determined by multiplying the effort multipliers associated with each of the 15 attributes.*

The Effort Adjustment Factor (EAF) is employed to enhance the estimates generated by the basic COCOMO model in the following expression:

➤ **Intermediate COCOMO Model equation:**

$$E = a * (KLOC)^b * EAF \text{ PM}$$

$$Tdev = c * (E)^d$$

Where,

- *E is effort applied in Person-Months*
- *KLOC is the estimated size of the software product indicate in Kilo Lines of Code*
- *EAF is the Effort Adjustment Factor (EAF) is a multiplier used to refine the effort estimate obtained from the basic COCOMO model.*
- *Tdev is the development time in months*
- *a, b, c are constants determined by the category of software project given in below table.*

The constant values a, b, c, and d for the Basic Model for the different categories of the software projects are:

Software Projects	a	b	c	d
Organic	3.2	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

3. Detailed COCOMO Model

- Detailed COCOMO goes beyond Basic and Intermediate COCOMO by diving deeper into project-specific factors.
- It considers a wider range of parameters, like team experience, development practices, and software complexity.

- By analyzing these factors in more detail, Detailed COCOMO provides a highly accurate estimation of effort, time, and cost for software projects. It's like zooming in on a project's unique characteristics to get a clearer picture of what it will take to complete it successfully.

CASE Studies and Examples

1. **NASA Space Shuttle Software Development:** NASA estimated the time and money needed to build the software for the Space Shuttle program using the COCOMO model. NASA was able to make well-informed decisions on resource allocation and project scheduling by taking into account variables including project size, complexity, and team experience.
2. **Big Business Software Development:** The COCOMO model has been widely used by big businesses to project the time and money needed to construct intricate business software systems. These organizations were able to better plan and allocate resources for their software projects by using COCOMO's estimation methodology.
3. **Commercial Software goods:** The COCOMO methodology has proven advantageous for software firms that create commercial goods as well. These businesses were able to decide on pricing, time-to-market, and resource allocation by precisely calculating the time and expense of building new software products or features.
4. **Academic Research Initiatives:** To estimate the time and expense required to create software prototypes or carry out experimental studies, academic research initiatives have employed COCOMO. Researchers were able to better plan their projects and allocate resources by using COCOMO's estimate approaches.

➤ Advantages of the COCOMO Model

1. **Systematic cost estimation:** Provides a systematic way to estimate the cost and effort of a software project.
2. **Helps to estimate cost and effort:** This can be used to estimate the cost and effort of a software project at different stages of the development process.
3. **Helps in high-impact factors:** Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
4. **Helps to evaluate the feasibility of a project:** This can be used to evaluate the feasibility of a software project by estimating the cost and effort required to complete it.

➤ Disadvantages of the COCOMO Model

1. **Assumes project size as the main factor:** Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.
2. **Does not count development team-specific characteristics:** Does not take into account the specific characteristics of the development team, which can have a significant impact on the cost and effort of a software project.
3. **Not enough precise cost and effort estimate:** This does not provide a precise estimate of the cost and effort of a software project, as it is based on assumptions and averages.

➤ **TLO 4.4**

➤ **Apply RMMM strategy in Identified risks for any software development problem:**

- To apply the [RMMM \(Risk Mitigation, Monitoring, and Management\)](#) strategy in software development, first identify potential risks, then assess their probability and impact, mitigate them with strategies like avoidance, reduction, or acceptance, and finally monitor those risks and their mitigation efforts continuously while having management and contingency plans in place to respond to unforeseen events.
- A risk management technique is usually seen in the software Project plan. This can be divided into Risk Mitigation, Monitoring, and Management Plan (RMMM). In this plan, all works are done as part of risk analysis. As part of the overall project plan project manager generally uses this RMMM plan.
- In some software teams, risk is documented with the help of a Risk Information Sheet (RIS). This RIS is controlled by using a database system for easier management of information i.e creation, priority ordering, searching, and other analysis. After documentation of RMMM and start of a project, risk mitigation and monitoring steps will start.

- **Risk Mitigation:**

This involves taking proactive steps to reduce the likelihood or negative impact of potential risks. For example, if a risk is that key team members might leave, mitigation could involve cross-training team members to share knowledge and reduce reliance on a single individual.

- It is an activity used to avoid problems (Risk Avoidance).
- Steps for mitigating the risks as follows.

1. Finding out the risk.
2. Removing causes that are the reason for risk creation.
3. Controlling the corresponding documents from time to time.
4. Conducting timely reviews to speed up the work.

- **Risk Monitoring:**

This is the ongoing process of tracking identified risks to see if their status has changed or if new risks have emerged. It ensures that mitigation strategies are still relevant and effective.

- It is an activity used for project tracking.
- It has the following primary objectives as follows.

1. To check if predicted risks occur or not.
2. To ensure proper application of risk aversion steps defined for risk.
3. To collect data for future risk analysis.
4. To allocate what problems are caused by which risks throughout the project.

- **Risk Management:**

- This encompasses the overarching process of handling risks, which includes identifying, analyzing, planning for, and responding to them as they arise.

- This task is done by Project manager when risk becomes reality and causes severe problems. If the project manager effectively uses project mitigation to remove risks successfully then it is easier to manage the risks.
- This shows that the response that will be taken for each risk by a manager. The main objective of the risk management plan is the risk register. This risk register describes and focuses on the predicted threats to a software project.

➤ **Example:**

Let us understand RMMM with the help of an example of high staff turnover.

1. Risk Mitigation:

To mitigate this risk, project management must develop a strategy for reducing turnover. The possible steps to be taken are:

- Meet the current staff to determine causes for turnover (e.g., poor working conditions, low pay, competitive job market).
- Mitigate those causes that are under our control before the project starts.
- Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
- Organize project teams so that information about each development activity is widely dispersed.
- Define documentation standards and establish mechanisms to ensure that documents are developed in a timely manner.
- Assign a backup staff member for every critical technologist.

2. Risk Monitoring:

As the project proceeds, risk monitoring activities commence. The project manager monitors factors that may provide an indication of whether the risk is becoming more or less likely. In the case of high staff turnover, the following factors can be monitored:

- General attitude of team members based on project pressures.
- Interpersonal relationships among team members.
- Potential problems with compensation and benefits.
- The availability of jobs within the company and outside it.

3. Risk Management:

Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality. Continuing the example, the project is well underway, and a number of people announce that they will be leaving. If the mitigation strategy has been followed, backup is available, information is documented, and knowledge has been dispersed across the team. In addition, the project manager may temporarily refocus resources (and readjust the project schedule) to those functions that are fully staffed, enabling newcomers who must be added to the team to "get up to the speed".

➤ **Drawbacks of RMMM:**

- It incurs additional project costs.
- It takes additional time.
- For larger projects, implementing an RMMM may itself turn out to be another tedious project.
- RMMM does not guarantee a risk-free project, in fact, risks may also come up after the project is delivered.

Important Question:

1. State different types of estimations in Software Projects.
2. Describe 4 P's of management spectrum.
3. Describe function point metrics for size estimation.
4. Describe RMMM strategy with example.
5. Use COCOMO model to calculate:
 - i) Efforts
 - ii) Development Time
 - iii) Average staff size
 - iv) Productivity.If estimation size of product is 400 KLOC using organic model.
6. Specify following cost directives of COCOMO:
 - i) Product Attributes
 - ii) Hardware Attributes
7. Describe following project cost estimation approaches:
 - i) Heuristic
 - ii) Empirical
8. Explain line of code metrics for size estimation.
9. Describe a cocomo and cocomo-II models